Introduction to Hashing

Imagine you have a huge bookshelf (like, Hogwarts library size). You've got a new book and need to find a spot for it and later on, search it quickly every time you need it. Instead of scanning the whole shelf, you use a magical spell that tells you exactly where to place or find it. This magical spell takes the book's title and gives you a specific location, like "4th shelf, 10th spot". But remember! Even if you slightly change the book's title, the spell gives a completely different spot.

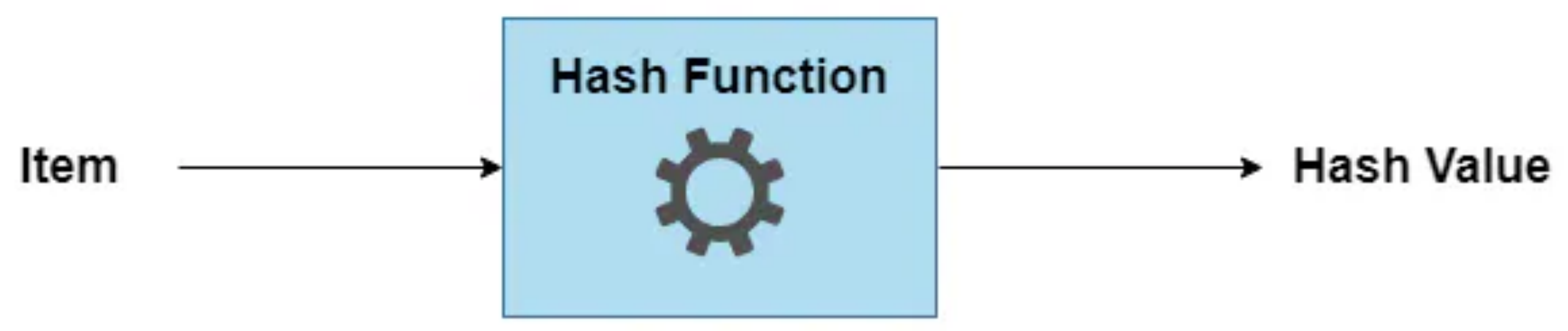
Now, let's map this to computer science:

Hashing: It's a technique that takes an input (or 'message') and returns a fixed-size string, which looks random. The output, known as the hash value, is unique (mostly) to the given input.

Hash Function: This is our "magical spell" that converts the input data (like our book title) into a fixed-length value.

However, no magic is perfect. If two different inputs give the same hash, that's called a **collision**. It's like our spell accidentally pointing to the same spot for two different books. Good hash functions make these super rare. We will discuss this in detail later on.

The following illustration depicts a generalized hashing scenario.



Hashing

But why hashing is important? Let's find out!

Why hashing?

Hashing is a great tool to quickly access, protect, and verify data. Here are a few of the common use cases of hashing:

1. **Quick Data Retrieval**: Hashing helps in accessing data super fast. With it, systems can quickly find a data piece without searching the whole database or list.
2. **Data Integrity Checks**: When downloading a file from the web, the site may provide you with a hash value for that file. If even a tiny portion of that file changes during the download, its hash will differ. By comparing the provided hash with the hash of the downloaded file, you can determine whether the file is exactly as the original, or if it was tampered with during the transfer
3. **Password Security**: Instead of storing actual passwords, systems store their hash. It's like locking the real magical item away and just keeping a hologram on display.
4. **Hash Tables**: Hashing is used in programming for efficient data structures like hash tables. It's like having organized shelves for our books where each item has its designated spot.
5. **Cryptography**: Some hash functions are used in cryptography to ensure data confidentiality and integrity. It's like a spell that only allows certain wizards to read a message.
6. **Data Deduplication**: If you're saving data, and you don't want duplicates, you can just compare their hashes. Same hash? It's the same data. It ensures you're not wasting space with repeated magical items.
7. **Load Balancing**: In big systems serving many users, hashes can be used to decide which server should handle a particular request. It's like deciding which magical portal to send a wizard based on their wand.

Hashing has numerous applications in several practical domains. However, this section tries to cover Hashing for defining and implementing a retrieval efficient data structure called Hash Table, which will be our next lesson.

← Previous
Solution: Merge Two Binary Trees (Medium)

Next →
Introduction To Hash Tables

☐ Mark as Completed